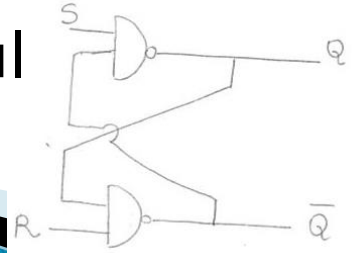
# Proiectare Logică

I CA & CB: Mariana Mocanu
mariana.mocanu@cs.pub.ro
I CC & CD: Costin Chiru costin.chiru@cs.pub.ro

## Curs 8: Circuite basculante bistabile. Multiplexoare

# Reminder (1)

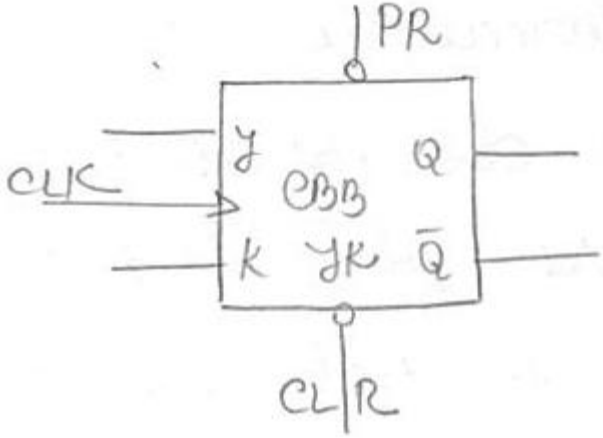
- Automate secvențiale
  - Folosirea organigramelor în sinteza circuitelor secvențiale – 10 pași
- CBB-uri
  - D – ecuația de stare: $Q^{t+1} = D^t$
  - R-S – ecuația de starea: $Q^{t+1} = !RQ^t + !RS$
  - Variabilele de stare definesc starea
  - Numărul variabilelor de stare e legat de numărul stărilor
- Exemple cu CBB-uri
  - interfornul




| R | S | Q | $\bar{Q}$ |
|---|---|---|---|
| 0 | 0 | nu se modif. st. ant | |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 0* | 0* |

stare neprevăzută

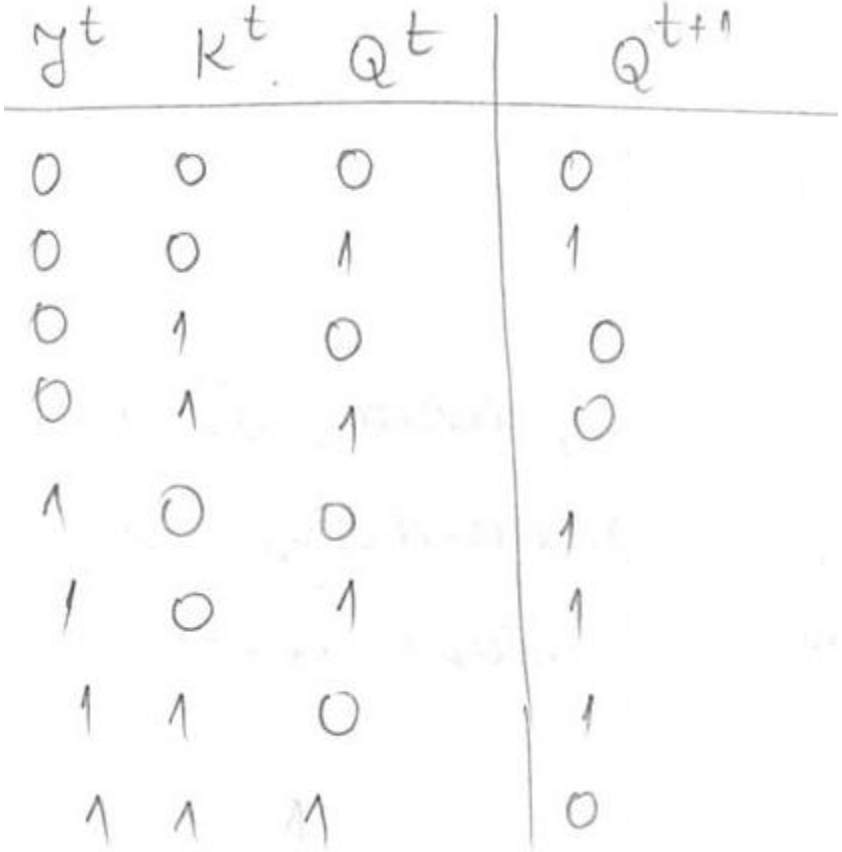
# Circuitul basculant bistabil JK



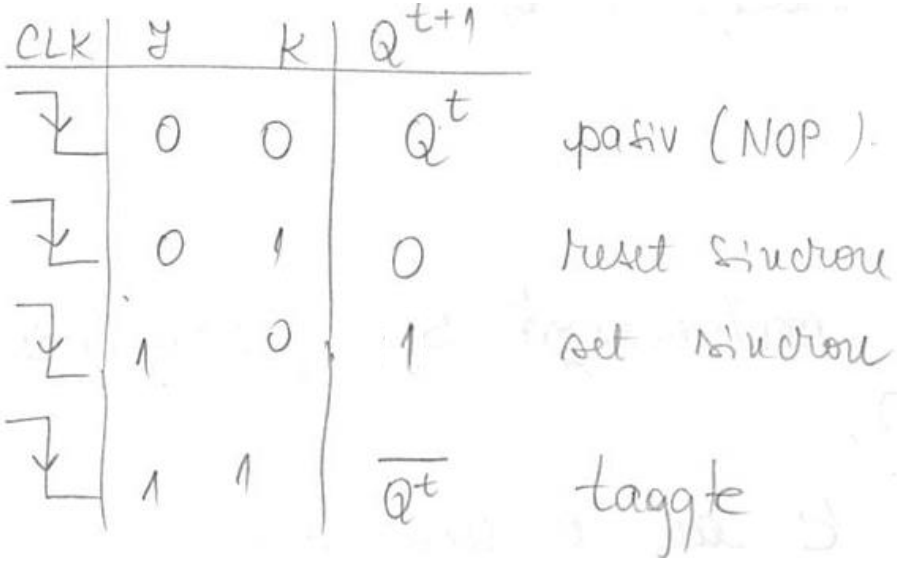

- 2 Intrări de date: J, K
- 1 Intrare de ceas: CLK
- 2 Ieşiri: Q și !Q
- 2 Intrări de comandă: R (clear)
  S (preset)
  - asincrone, prioritare

- Implementarea fizică constă în 2 circuite pe pastilă
- Are funcționalitate asincronă precum celelalte CBB-uri
- Are un comportament derivat din funcționarea RS (unde aveam comportament neplăcut din cauza stării de incontrolabilitate)
- A apărut pentru a corecta problema de la RS (pentru ∀ combinație de intrare, se știe în ce stare trece CBB JK)

# CBB JK – tabelul de adevăr



| $J^t$ | $K^t$ | $Q^t$ | $Q^{t+1}$ |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

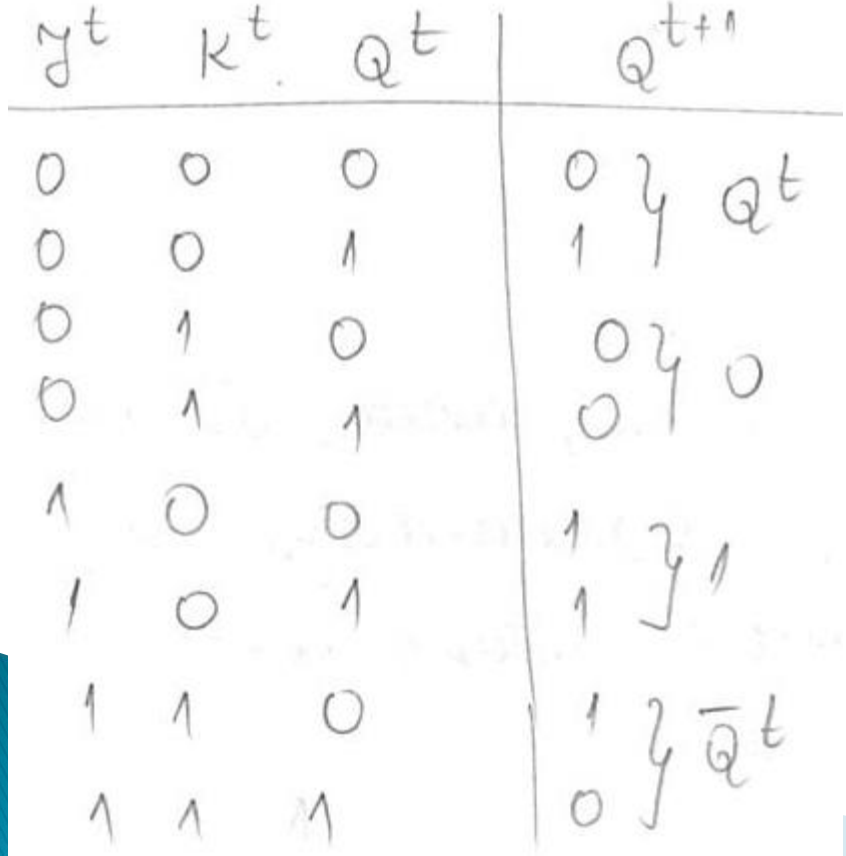


| CLK | J | K | $Q^{t+1}$ | |
|---|---|---|---|---|
| ↓ | 0 | 0 | $Q^t$ | pasiv (NOP) |
| ↓ | 0 | 1 | 0 | reset sincron |
| ↓ | 1 | 0 | 1 | set sincron |
| ↓ | 1 | 1 | $\overline{Q^t}$ | toggle |

- Trecem la varianta doar în funcție de J și K:
- Tabelul de adevăr este identic cu cel de la CBB RS, cu excepția J = K = 1, când ieșirea își modifică starea

# CBB JK – ecuația de stare următoare

- Pe baza tabelului de adevăr încercăm să determinăm ecuația de stare următoare → completăm diagrama Karnaugh corespunzătoare




| Q \ JK | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 |

$$Q^{t+1} = \bar{K}^t Q^t + J^t \cdot \bar{Q}^t$$

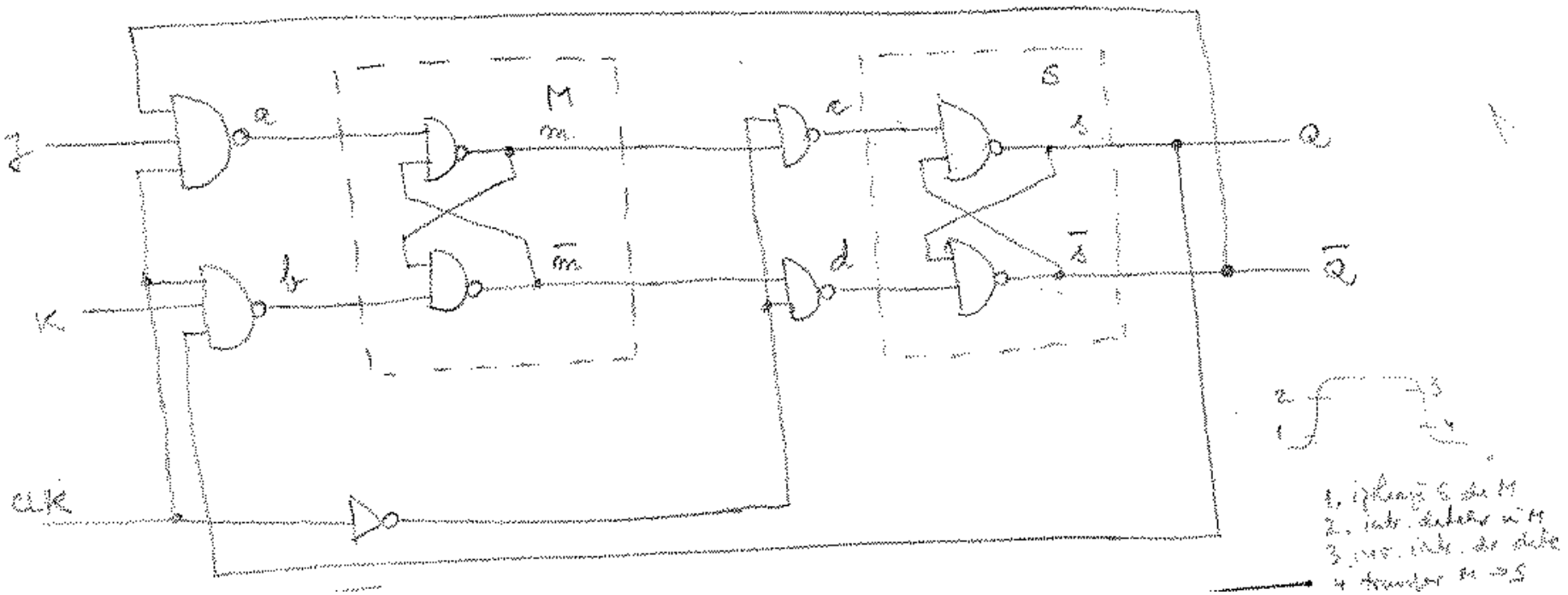
A 3-a ecuație pe care trebuie să o memorați

# CBB – determinare J și K

- Cel mai răspândit tip de CBB
- Ca la RS, în general se cunoaște funcționarea circuitului și se dorește identificarea elementelor de circuit necesare pentru a se implementa J și K. Acestea se determină pe baza tabelului

| $Q^t$ | $Q^{t+1}$ | J | K |
|---|---|---|---|
| 0 | 0 | 0 | * |
| 0 | 1 | 1 | * |
| 1 | 0 | * | 1 |
| 1 | 1 | * | 0 |

# CBB JK – implementare (1)



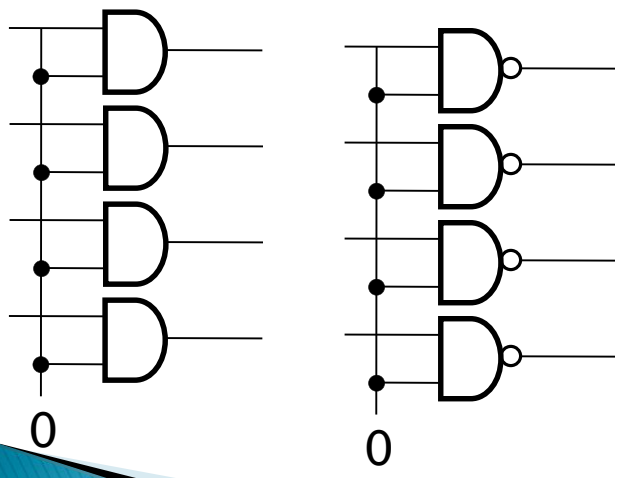

- Circuit secvențial de ce ordin?
- O luăm pe bucățele:
  - Zonele punctate reprezintă CBB-uri elementare (de ordin 1)
  - Cum sunt legate? În serie →ordin 1
  - DAR! Observăm cele 2 conexiuni de reacție (!Q – J & Q – K)
  - → Ordinul este 2 pentru că are în bucla de reacție CBB de ordin 1

# CBB JK – implementare (2)

- Avem 2 părți: una de master (M) și una de slave (S)
- Am vorbit în trecut de sincronizarea
  - Pe front
  - Master-slave (în 4 timpi). Care sunt acțiunile din cei 4 timpi?
    - 1. decuplarea M de S,
    - 2. intrarea datelor în M,
    - 3. invalidarea intrărilor,
    - 4. trecerea datelor din M în S
    - Această sincronizare are ca scop protejarea circuitului de momente ce țin de timpii de propagare, timpii de stabilizare, momentele când tensiunea este între nivelele de 0 și 1, etc.

# CBB JK – sincronizare (1)

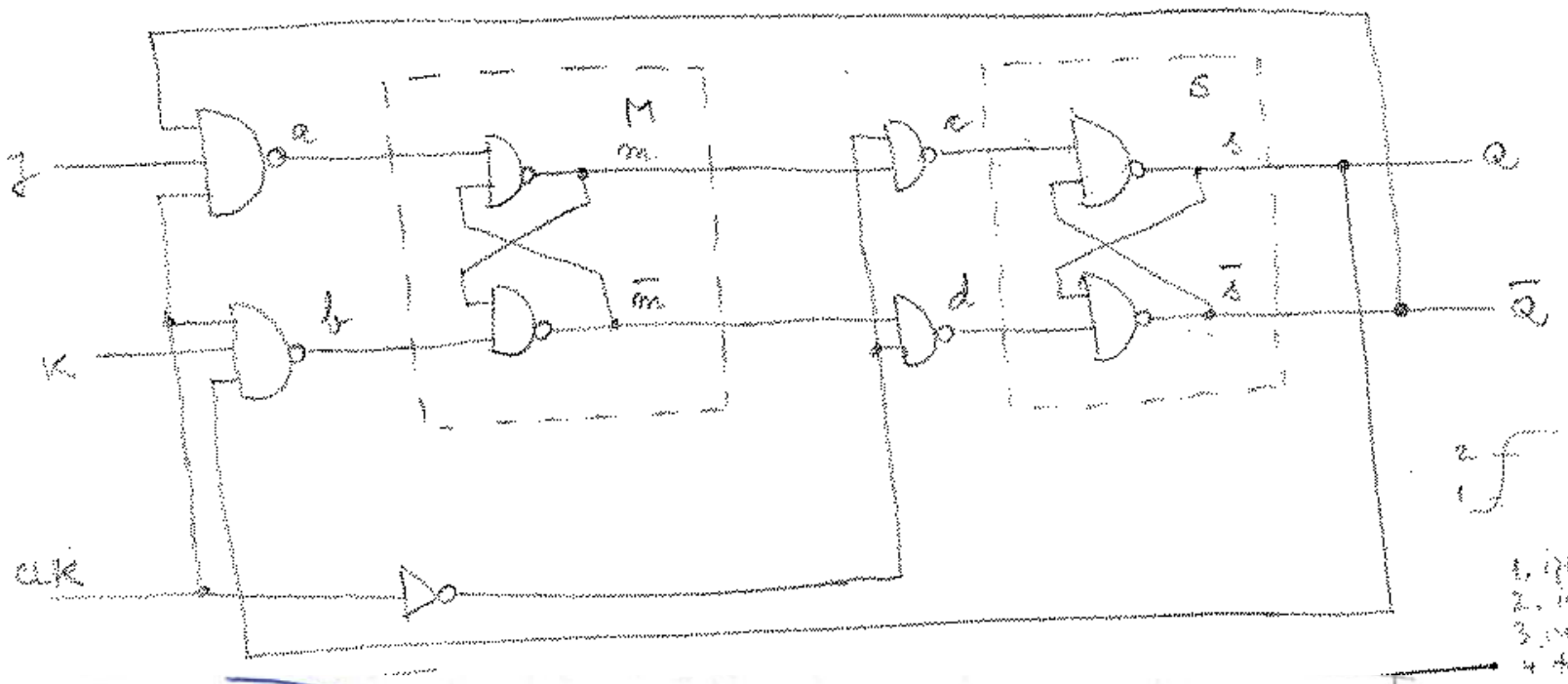
- Ce înseamnă invalidarea intrărilor? Cum se face?
- Porțile ȘI pot bloca aplicarea datelor utile prin simpla aplicare a unei valori 0. La NAND e la fel, ieșirea nefiind 0, ci 1.




- Este foarte folosită pentru că protejează circuitul și este foarte simplă (ieftină, fără erori)
- Așa se întâmplă cu nivelul a-b / c-d de NAND-uri din CBB JK

# CBB JK – sincronizare (2)

- Care sunt datele utile?
- J & K
- Cum blocăm circuitul?
- Cu ajutorul CLK
- Cum adică?
- O dată e valid un nivel, o dată celălalt
- Cum se face asta?
- Cu ajutorul inversorului → el asigură decuplarea




$$S = \overline{c \cdot \overline{Sd}} = \bar{c} + Sd$$

$$m = \overline{a \cdot \overline{bm}} = \bar{a} + bm$$

$$c = \overline{m \cdot \overline{clk}}$$

$$d = \overline{\overline{m} \cdot \overline{clk}}$$

$$a = \overline{J \cdot clk \overline{Q}}, \quad \overline{Q} = \bar{S}$$

$$b = \overline{K \cdot clk S}$$

$\Rightarrow$

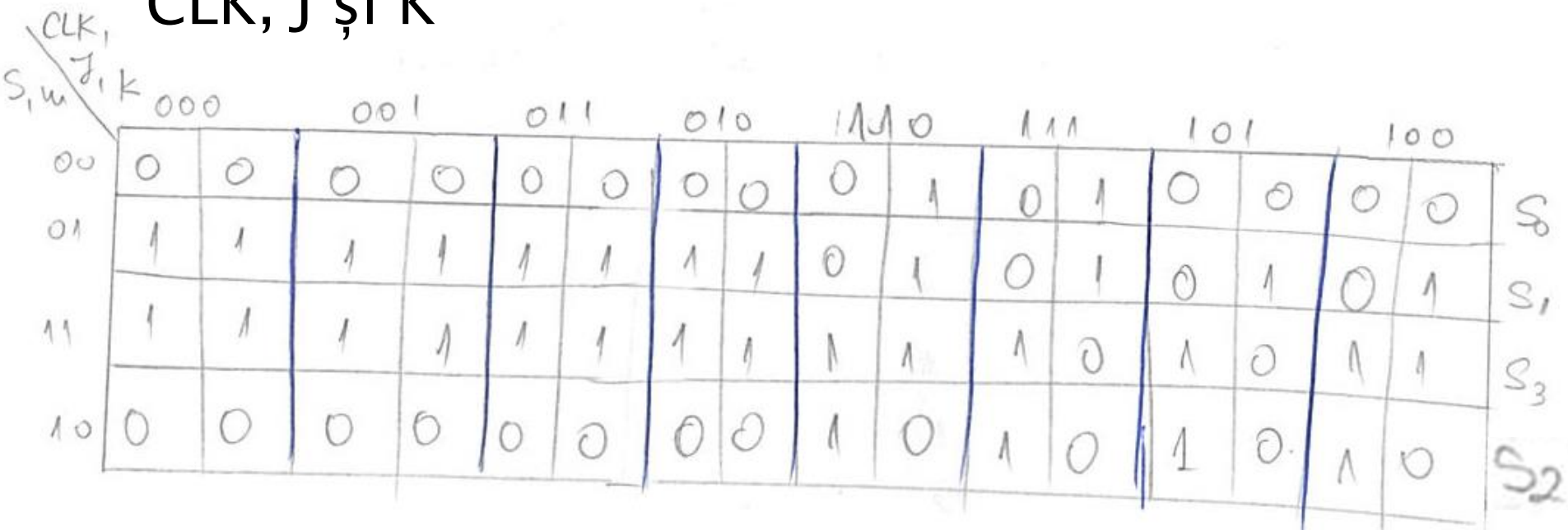
Determinăm formulele pentru s și m:

$$S = m \cdot \overline{clk} + mS + clk \cdot S$$

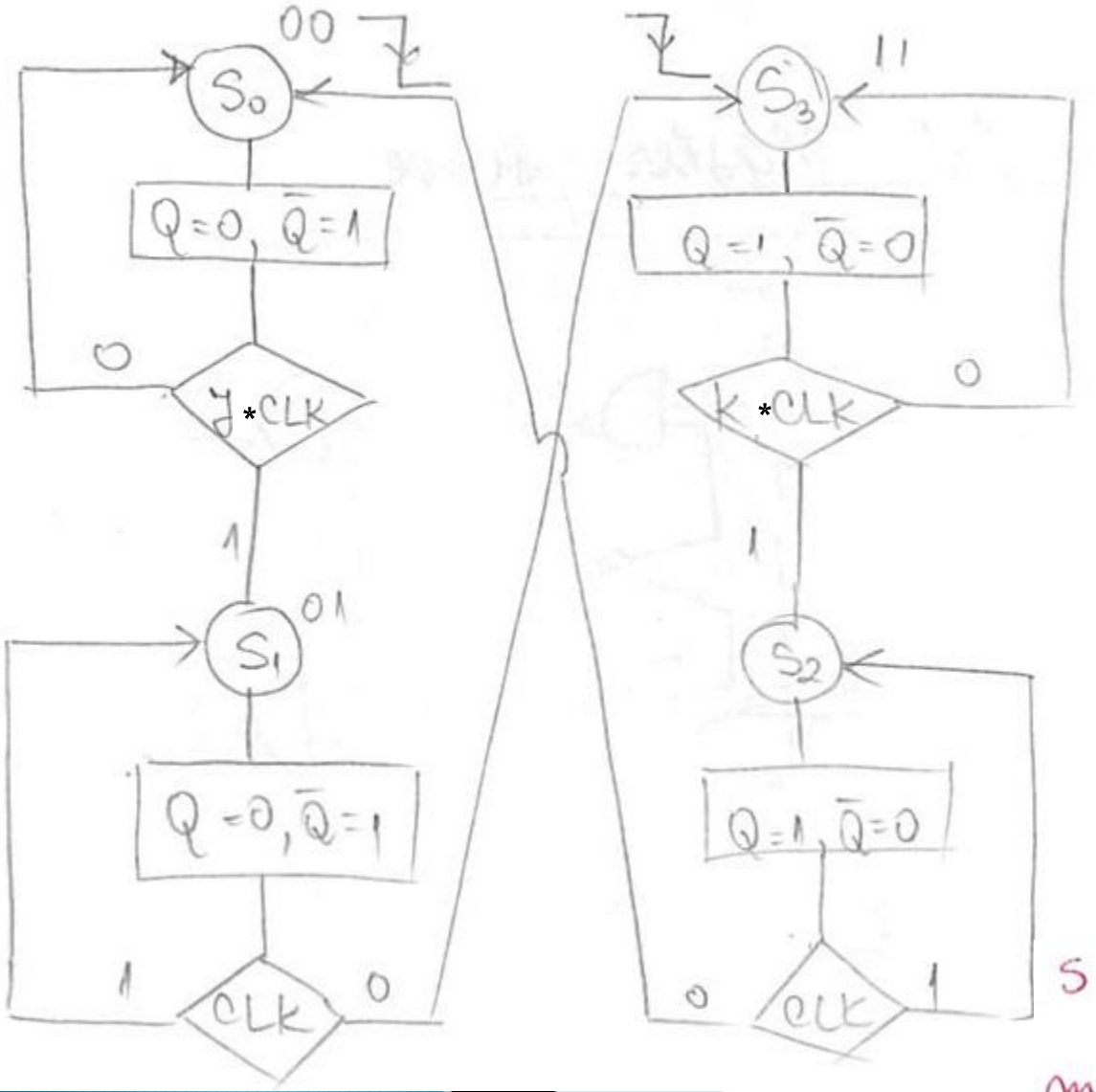
$$m = J \cdot clk\,\bar{S} + \bar{K}m + \overline{clk}m + \bar{S}m$$

# CBB JK – funcționare

- Putem considera m și s ca fiind variabilele de stare ale CBB-urilor elementare → circuitul are 4 stări corespunzătoare acestor variabile de stare.
- Construim diagrama Karnaugh folosind s, m, CLK, J și K



# CBB JK – Organigrama




- Cele 4 momente de la sincronizarea master-slave corespund celor 4 stări.
- Temă: care moment corespunde cărei stări? + Justificare.

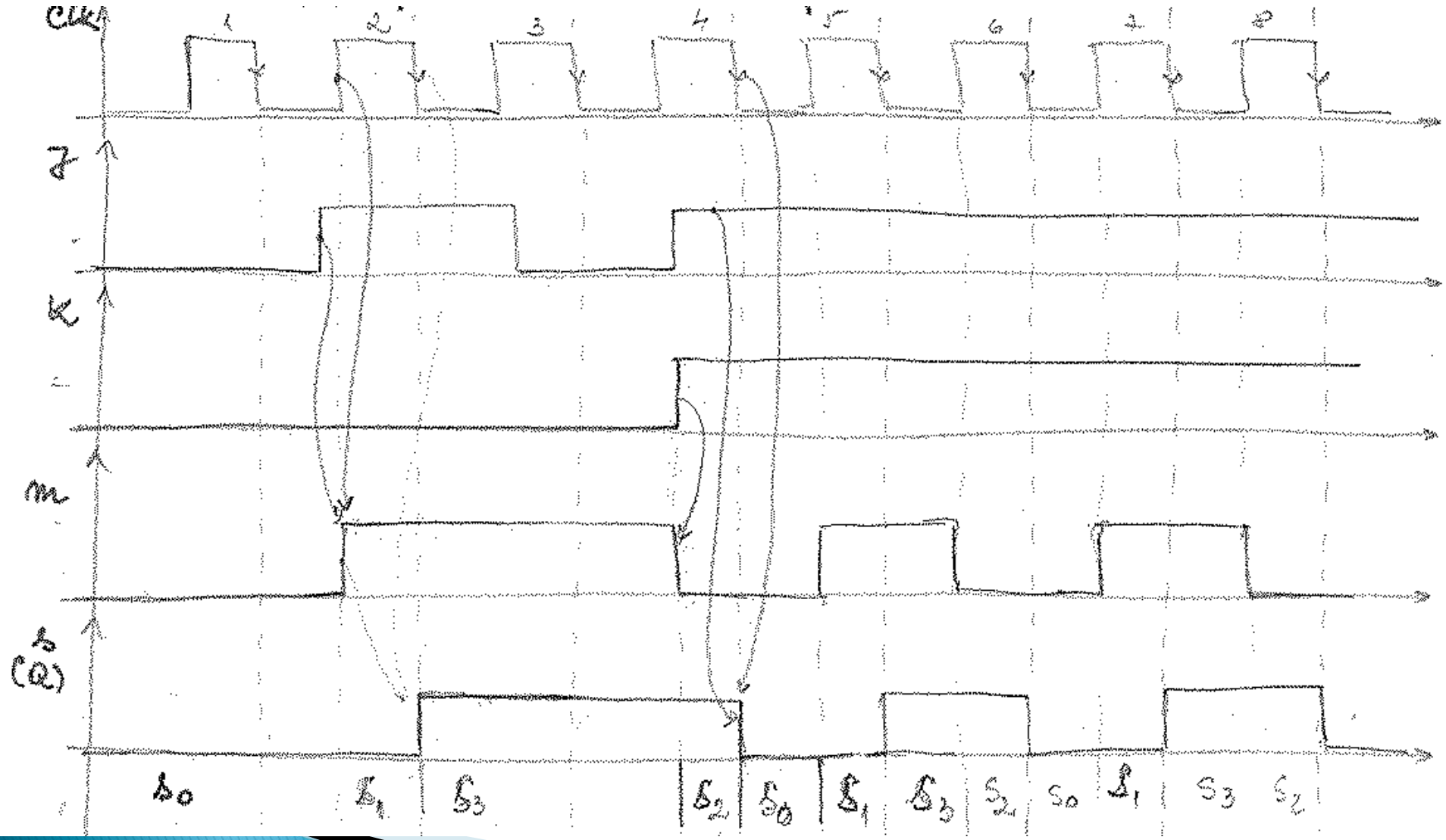
$$S = m \cdot \overline{CLK} + mS + CLK \cdot S$$

$$m = J \cdot CLK\,\bar{S} + \bar{K}m + \overline{CLK}m + \bar{S}m$$

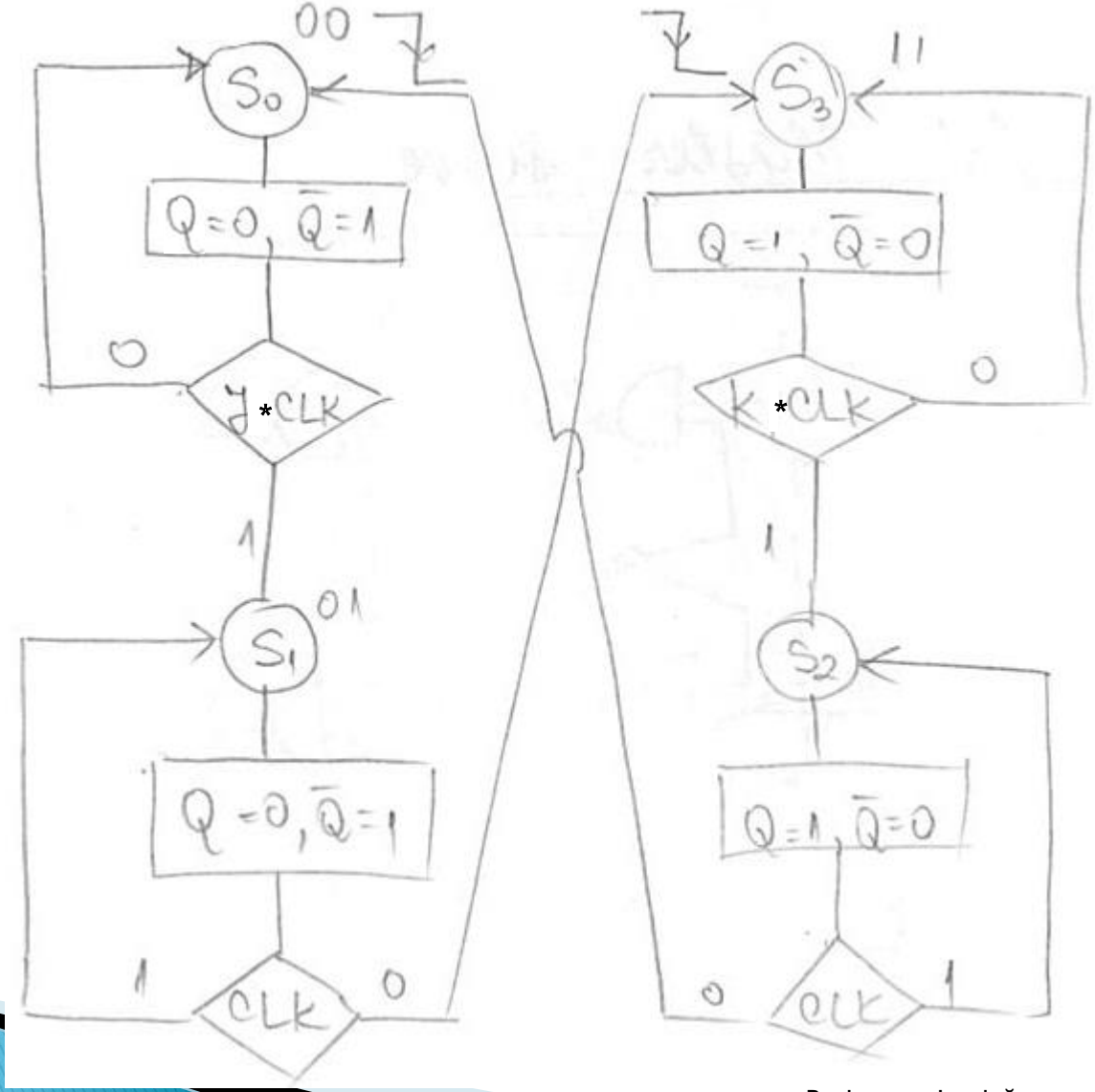
# CBB JK – funcționare

- Tranziția mărimilor de ieșire se manifestă pe căderea impulsului de ceas → funcționarea circuitului este sincronă pe frontul descrescător al semnalului CLK
- Când CLK = 0, circuitul este fie în $S_0$, fie în $S_3$, când CLK = 1, circuitul este fie în $S_1$, fie în $S_2$
- J controlează $S_0$ a circuitului (cât timp J = 0, circuitul rămâne blocat în $S_0$) → J controlează trecerea lui Q din 0 în 1
- K controlează $S_3$ a circuitului (cât timp K = 0, circuitul rămâne blocat în $S_3$) → K controlează revenirea lui Q din 1 în 0
- Obs: J și K au funcționalitate similară cu S și R, dar nu pot determina funcționarea circuitului în lipsa ceasului (acțiunea lor e de tip sincron)

# CBB JK – Diagrama de semnal



# CBB JK – Organigrama

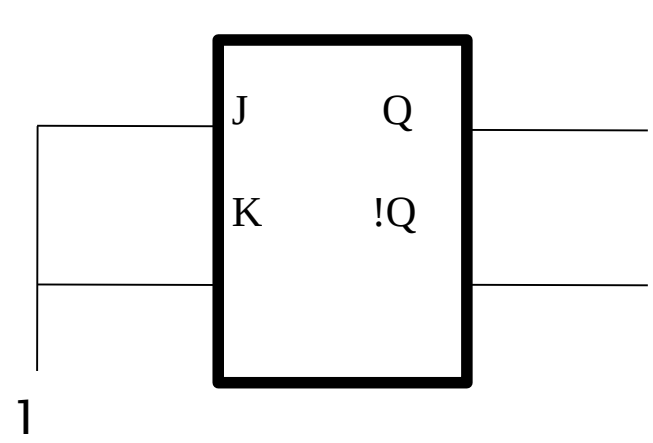


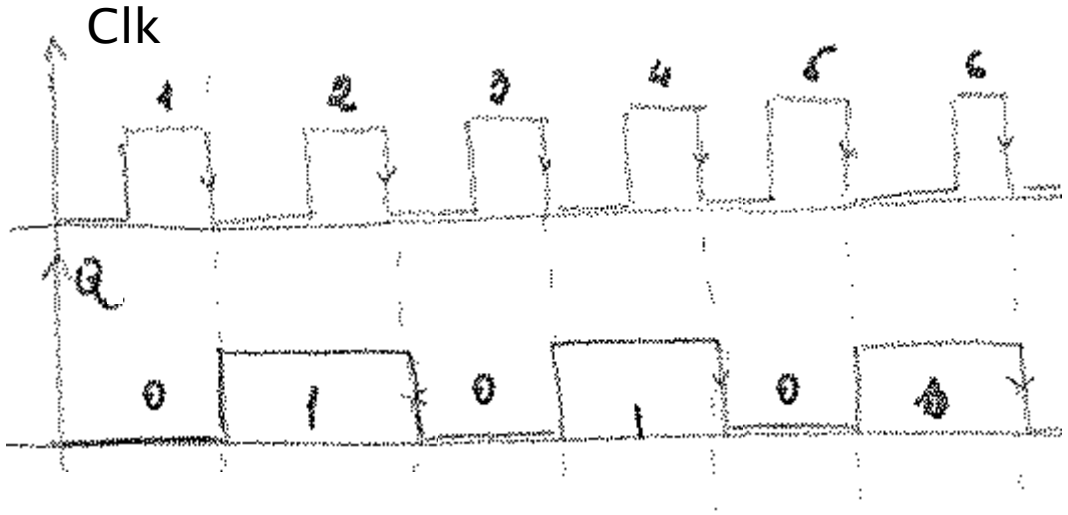
# CBB JK –explicații

- Circuitul M captează informația utilă pe durata activă a lui CLK (= 1)
- Circuitul S copiază informația din M în momentul trecerii ceasului în 0
- În practică, circuitul este prevăzut și cu comenzi asincrone (preset și clear) active pe 0:

| PR | R | J | K | CLK | $Q^{t+1}$ |
|---|---|---|---|---|---|
| 1 | 1 | * | * | ↓ | ca mai devreme |
| 0 | 1 | – | – | – | 1 set (activare) |
| 1 | 0 | – | – | – | 0 reset |
| 0 | 0 | – | – | – | ✗ |

# CBB JK – aplicație (1)



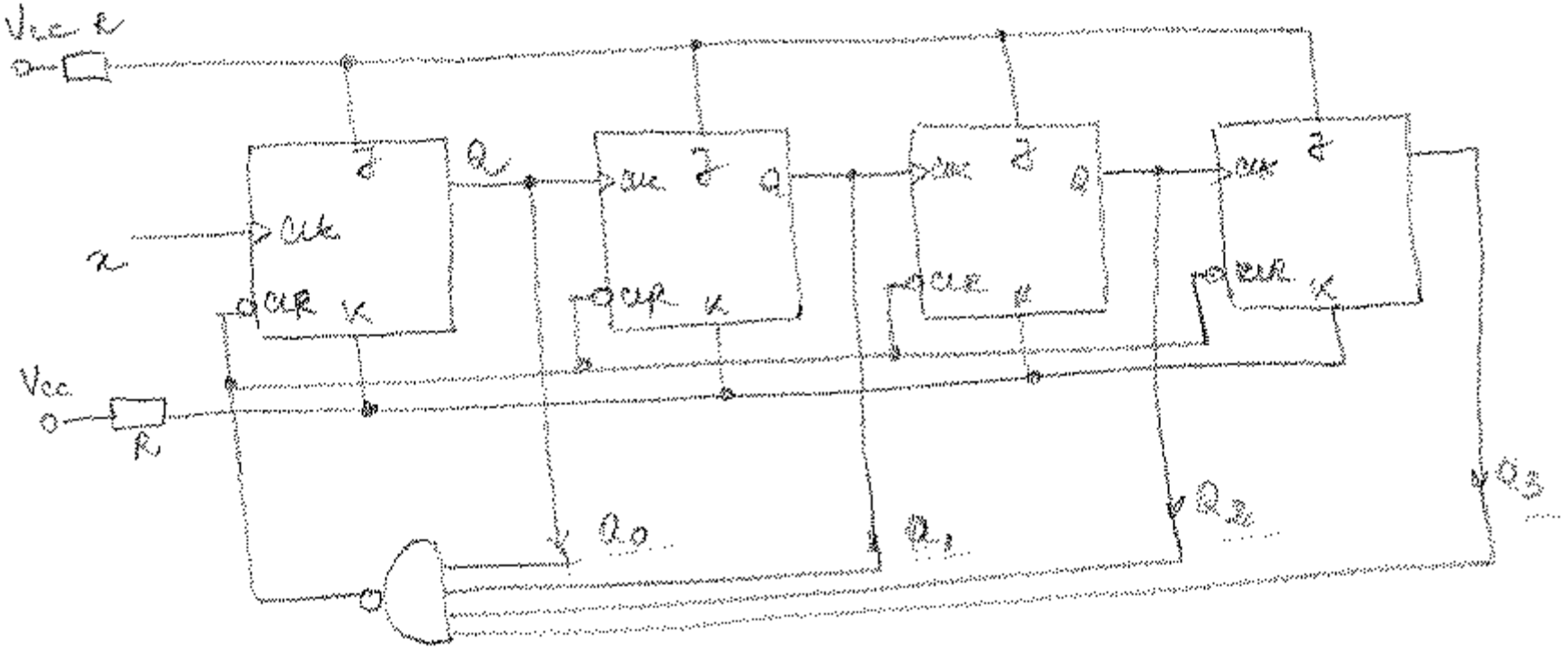




- Ce face?
- Inversează valoarea lui Q: $Q^{t+1} = !\,Q^{t}$
- Divizor de frecvență
- Am mai intâlnit?
- Da, la CBB D

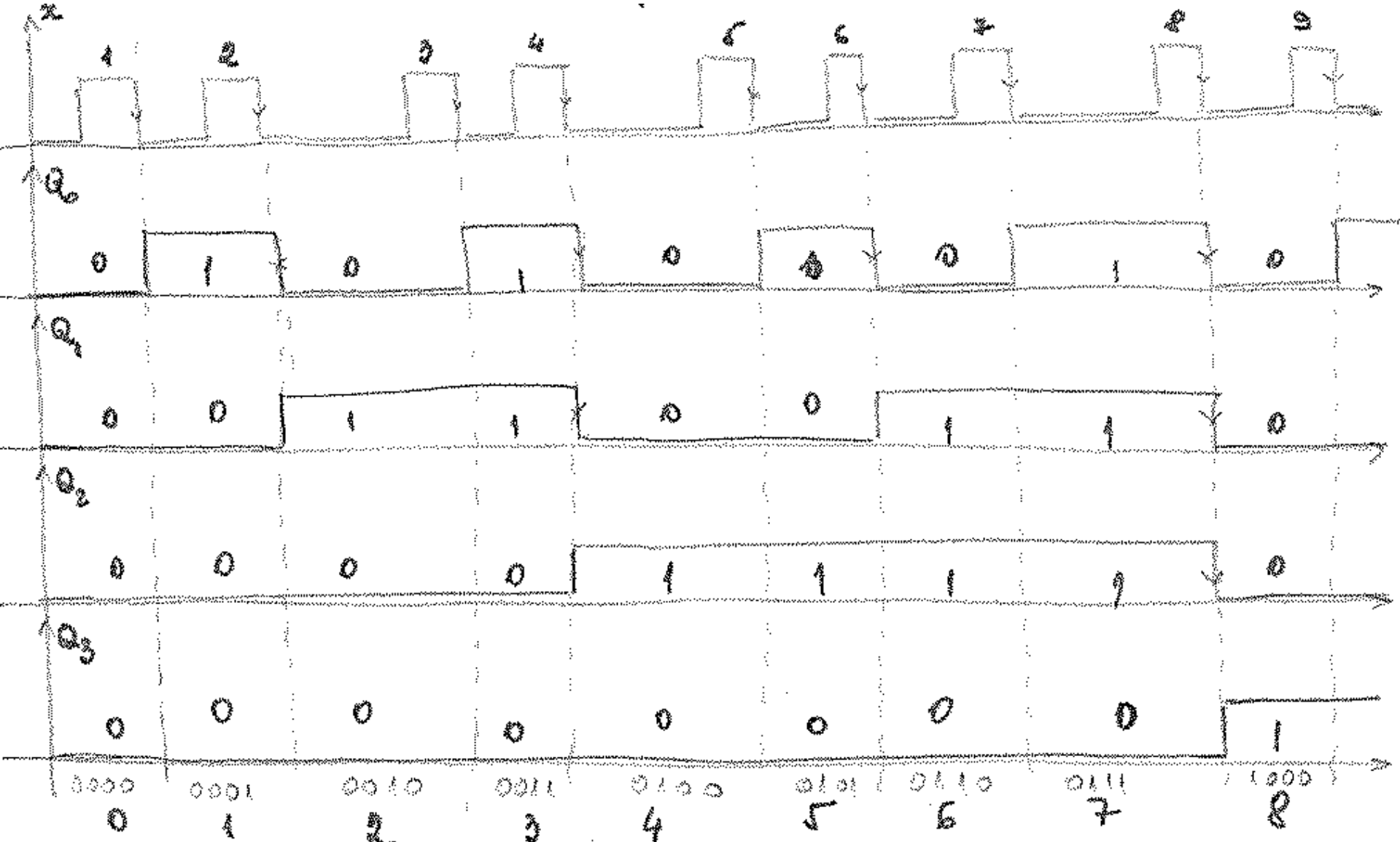
$$Q^{t+1} = \bar{K}^{t} Q^{t} + J^{t} \cdot \bar{Q}^{t}$$

# CBB JK – aplicație (2) – Numărător pe 4 biți

- Schemă clasică de numărător
- Când face reset? Când toate Q-urile sunt 1
- Numărător ciclic → după ce ajunge la 1111 trece în 0



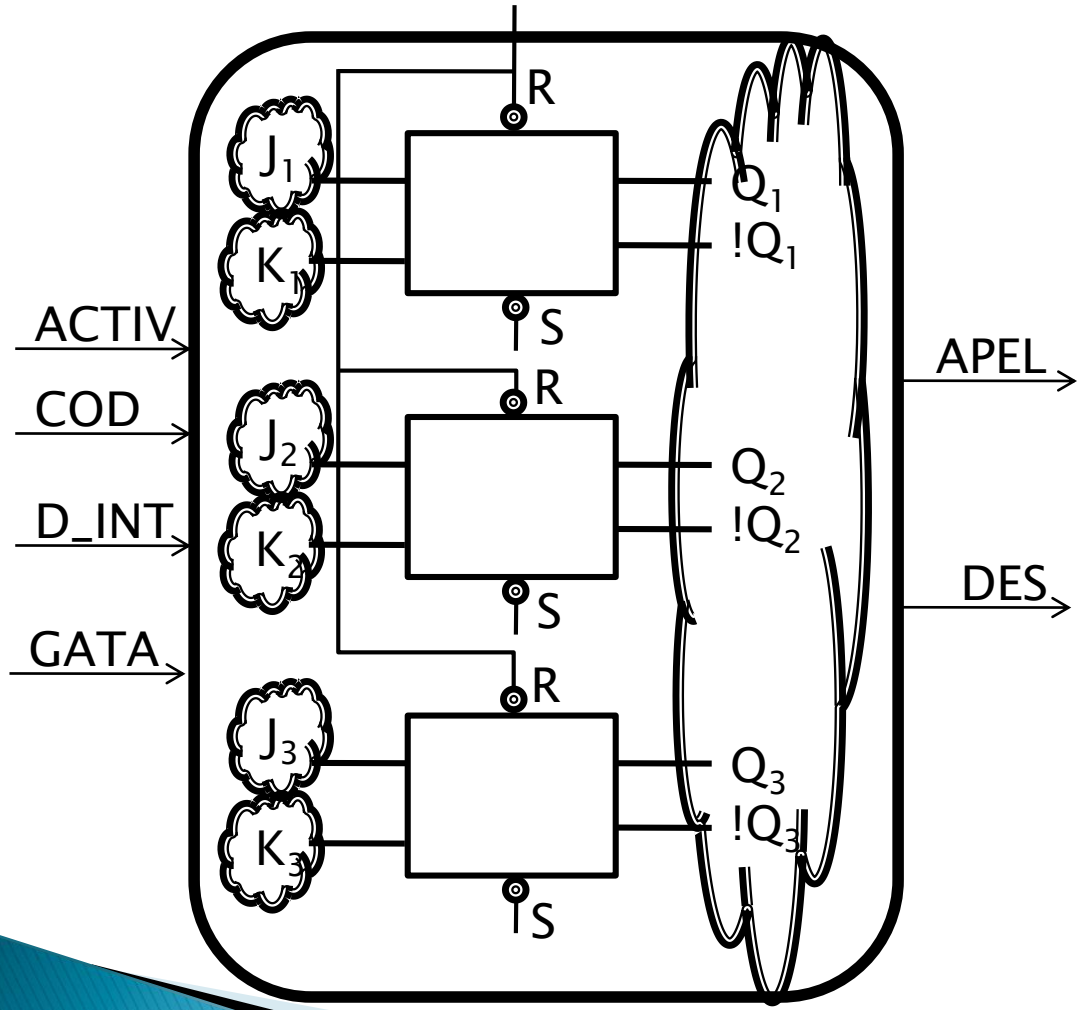
# Numărător – Diagrama de semnal



# Numărător – detalii

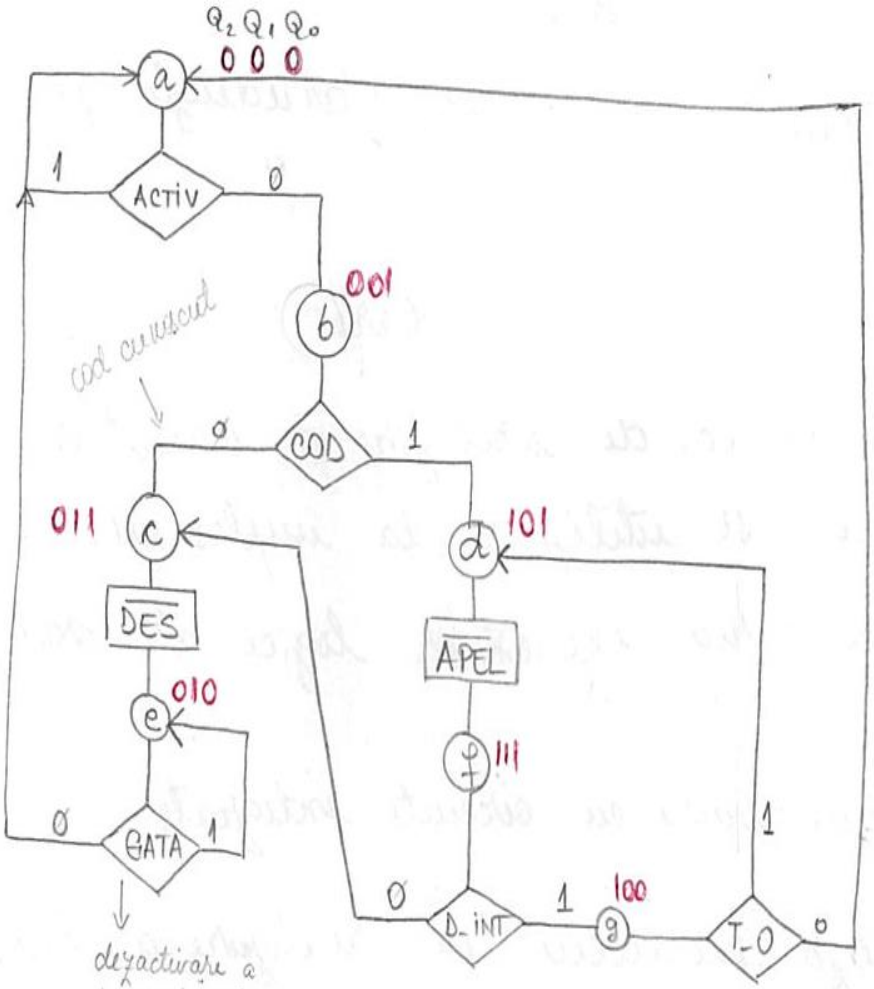
- Care este cel mai semnificativ bit și care cel mai puțin semnificativ bit?
- MSB: $Q_3$, LSB: $Q_0$
- În funcție de ce punem în poarta de jos se modifică valoarea maximă până la cât numără
- Ex: ca să fac numărător decimal, trebuie să neg intrările de la $Q_0$ și $Q_2$
- Putem să îl extindem la mai mulți biți?
- Dezavantaj: fiecare circuit JK introduce întârzieri și dacă semnalul x este prea rapid se pierd din semnale

# CBB JK – aplicație (3) – Interfon




- Similar cu CBB de tip D, trebuie să detectăm ce punem pe intrări (J,K) și cum combinăm ieșirile (Q,!Q)
- La CBB D era foarte simplu de determinat intrările ($Q^{t+1} = D^t$)
- La CBB JK trebuie să împărțim $Q^{t+1}$ în intrarea lui J și a lui K

# Interfon – reminder




| $Q_2^t$ | $Q_1^t$ | $Q_0^t$ | $Q_2^{t+1}$ | $Q_1^{t+1}$ | $Q_0^{t+1}$ |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | !ACTIV |
| 0 | 0 | 1 | COD | !COD | 1 |
| 0 | 1 | 0 | 0 | GATA | 0 |
| 0 | 1 | 1 | 0 | 1 | 0 |
| 1 | 0 | 0 | T_O | 0 | T_O |
| 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | * | * | * |
| 1 | 1 | 1 | D_INT | ! D_INT | ! D_INT |

# Interfonul (3)

- Cum împărțim?

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 0 | * | T_O |
| 1 | COD | 0 | D_INT | 1 |

$Q_2 = Q_2!Q_1Q_0 + T_O*Q_2!Q_1 + COD*!Q_1Q0 + D_INT*Q_2Q_0$

- Știm că $Q^{t+1} = J!Q^t + !KQ^t$
- Ce se întâmplă cu $Q^{t+1}$ dacă $Q^t = 0$? $Q^{t+1} = J$
- Ce se întâmplă cu $Q^{t+1}$ dacă $Q^t = 1$? $Q^{t+1} = !K$
- Citim invers: $J = Q^{t+1}|_{Q^t = 0}$ și $!K = Q^{t+1}|_{Q^t = 1}$
- $J = Q^{t+1}$ dar numai partea corespunzătoare lui $Q^t = 0$ → unde e $Q^t = 0$ ? Pentru $Q_2$ = stânga

# Interfonul (4)

- Arătăm că ecuația de stare exprimă o combinare a 2 componente (când $Q^t = 0$ și $Q^t = 1$).
- Cele 2 situații nu se suprapun și acoperă întreaga diagramă → putem defini pe J și K fără ca cele 2 componente să se influențeze una pe alta →
- Am mai întâlnit asta? Da! Superpoziția de la diagrame Karnaugh cu variabile externe
- → ca să știm ce punem pe intrările CBB JK lucrăm cu diagrama de stare următoare și o spargem în 2: zona pentru J și zona pentru !K

# Interfonul (5)

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 0 | * | T_O |
| 1 | COD | 0 | D_INT | 1 |

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 0 | * | * |
| 1 | COD | 0 | * | * |

$J_2 = COD!Q_1Q_0$

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | * | * | * | T_O |
| 1 | * | * | D_INT | 1 |

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | * | * | * | !T_O |
| 1 | * | * | !D_INT | 0 |

$K_2 = !T_O\ Q_0 + !D_INT\ Q_1$

- Pornim de la diagrama de stare
- Care e zona pentru J2?
- Primele 2 coloane → restul nu ne interesează → punem *
- La fel pentru !K
- Putem face direct diagrama lui K prin negarea fiecărei căsuțe în parte → de obicei așa se face ca să nu greșiți!!!

# Interfonul (6)

- Cum se face împărțirea pentru $J_1$ și $K_1$?
- E la fel ca pentru $Q_2$?

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | Gata | * | 0 |
| 1 | !COD | 1 | !D_INT | 1 |

$$Q_1 = Q_2!Q_1Q_0 + !Q_2Q_1Q_0 + !COD*!Q_2Q0 + Gata*Q_1!Q_0 + !D_INT*Q_2Q_0$$

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | * | * | 0 |
| 1 | !COD | * | * | 1 |

$$J_1 = Q_2Q_0 + !COD*Q0$$

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | * | !Gata | * | * |
| 1 | * | 0 | D_INT | * |

$$K_1 = !Gata*!Q_0 + !D_INT*Q_2$$

# Interfonul (7)

- Cum se face împărțirea pentru $J_0$ și $K_0$?

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | !ACTIV | 0 | * | T_O |
| 1 | 1 | 0 | !D_INT | 1 |

$$Q_0 = !Q_1Q_0 + !ACTIV*!Q_2!Q_1 + T_O*Q_2!Q_1 + !D_INT*Q_2Q_0$$

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | !ACTIV | 0 | * | T_O |
| 1 | * | * | * | * |

$$J_0 = !ACTIV*!Q_2!Q_1 + T_O*Q_2$$

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | * | * | * | * |
| 1 | 0 | 1 | D_INT | 0 |

$$K_0 = !Q_2Q_1 + D_INT*Q_1$$

# Interfonul (8)

- OBSERVAȚIE: prin spargerea lui $Q^{t+1}$ în zonele corespunzătoare lui J și K avem niște formule care ne duc la o complexitate mai mică → circuitele pe intrările lui J și K sunt mai simple decât ce puneam pe intrarea lui D, dar trebuie să fim atenți cum facem împărțirea (și să nu uităm să negăm)

# Interfonul (9)

- Ieșirile circuitului rămân la fel ca în cazul CBB D (ca să fie active pe 0, trebuie negate după minimizare):

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 0 | * | 0 |
| 1 | 0 | 1 | 0 | 0 |

$DES = !Q_2Q_1Q_0$

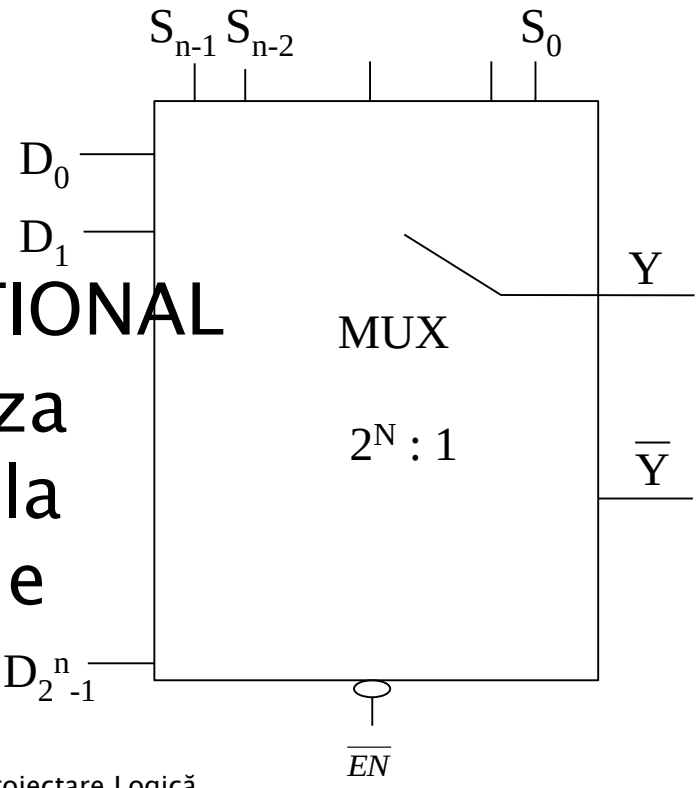
| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | 0 | * | 0 |
| 1 | 0 | 0 | 0 | 1 |

$APEL = Q_2!Q_1Q_0$

- Acum putem trece la implementarea circuitului cu CBB de tip JK

# Multiplexoare

- Circuite care selectează dintr-o multitudine de intrări, pe baza unei condiții, o anumită valoare pe care o trimite la ieșire
- L-ați învățat la laborator.
- De ce îl facem acum?
- Este circuit secvențial?
- Nu! ESTE CIRCUIT COMBINAȚIONAL
- Poate fi folosit pentru a realiza circuitele combinaționale de la intrările circuitelor secvențiale

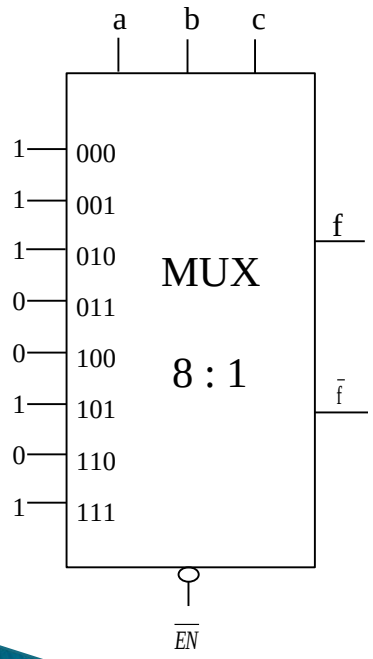


# Multiplexoare (2)

- ***Sunt circuite combinaționale*** și vom arăta cum le putem folosi pentru a le combina cu circuitele secvențiale
- Atenție: la examen am observat diverse situații în care apare MUX și dispare CBB, ca și cum MUX ar putea să implementeze partea de secvențial → Nu e corect!
- Ce facem de fapt cu MUX? Realizăm intrările CBB-urilor în loc sau în combinație cu porți

# Multiplexoare (3)

- Exemplu:
- f(a,b,c) = a!bc + !a!bc + abc + !a!b!c+!ab!c

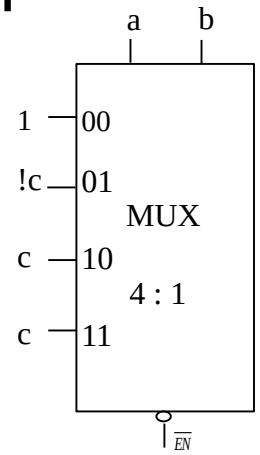



- Cum rezolvăm?
- Scriem forma generală a unei funcții ca sumă de monoame:
- $f(a,b,c) = c_0$* !a!b!c + $c_1$ * !a!bc + ... + $c_7$ * abc
- De ce am scris funcția așa?
- Ca să putem identifica coeficienții și să îi punem pe intrările MUX-ului

# Multiplexoare (4)

$f(a,b,c) = a!bc + !a!bc + abc + !a!b!c + !ab!c$

- E greu de realizat asta?
- Nu – iau un MUX, identific coeficienții și aplic pe intrări valori elementare.
- Super, dar cu cât e mai mare un MUX, cu atât e mai scump → vreau să implementez aceeași funcție cu MUX 4:1
- Se obține același rezultat dacă pun a,b sau a,c sau b,c pe condiții?
- Nu!
- $f(a, b) = c_0*!a!b + c_1*!ab + c_2*a!b + c_3*ab$
- Identific coeficienții și completez MUX
- Pot trece și la MUX 2:1 în același fel!
- Atenție mare! La examen să puneți variabilele de selecție și ordinea în care sunt puse datele



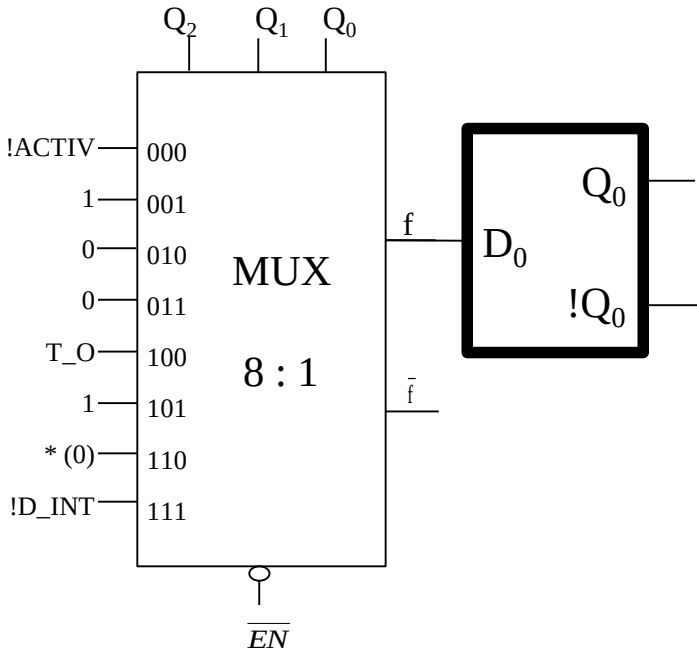
# Interconectare MUX – CBB

- La examen, în enunțul problemelor pot varia doar tipul de CBB și modul de realizare a circuitelor combinaționale.
- Ex: implementați Qi folosind
  - CBB D și porți (eventual de un anume fel);
  - CBB D și MUX (de un anume tip);
  - JK și porți / MUX
  - JK și porți pentru J, respectiv MUX pentru K
- În cei 10 pași am delimitat ecuațiile de stare următoare de implementarea cu CBB-uri (la tip D era același lucru, dar la JK / RS trebuie să împărțim în 2)
- Care e cea mai simplă implementare? D + MUX 8:1

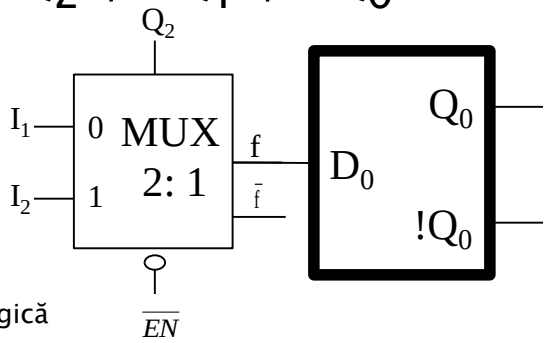
# CBB D – MUX

- Care e cea mai simplă implementare?
- D + MUX 8:1
- Ex:

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | !ACTIV | 0 | * | T_O |
| 1 | 1 | 0 | !D_INT | 1 |




- Altă implementare: D+MUX 2:1
- Întrebare: câte MUX 2:1? (1/7)
- Întrebare2: ce variabile de selecție? $Q_2$ / $Q_1$ / $Q_0$?
- Temă: Exersați toate combinațiile posibile (MUX 2:1, MUX 4:1, var sel)

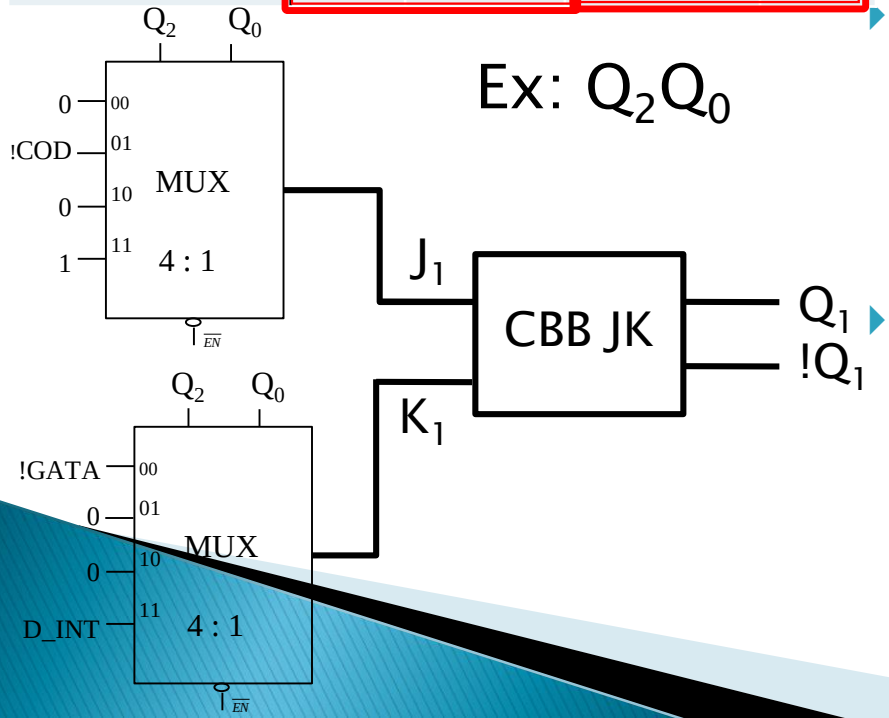


# CBB JK – MUX

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | 0 | * | * | 0 |
| 1 | !COD | * | * | 1 |

| $Q_0$ \ $Q_2Q_1$ | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 0 | * | !Gata | * | * |
| 1 | * | 0 | D_INT | * |

Ex: $Q_2Q_0$




- În practică, pentru cele 2 MUX se aleg aceleași variabile de selecție dar la examen le putem cere diferite (în scop didactic)
- Atenție: Pe condițiile de la MUX nu se pun variabile externe (ex: Gata, Cod etc.), ci doar variabile de stare
- Mereu ne ocupăm de partea secvențială și apoi de cea combinațională. Începem cu CBB și apoi MUX → reverse engineering
- Verificare: nu pot apare variabilele de selecție pe intrări → termenul minimizat trebuie să conțină termenul de pe intrare + variabilele de selecție în combinația dată

# ÎNTREBĂRI?